



이제 직접 계산을 돌려봅니다

KISTI 누리온 슈퍼컴퓨터에서 Silicon(Si)을 예제로,
환경 설정부터 vc-relax, SCF, 밴드, DOS까지
복사-붙여넣기로 따라할 수 있게 만든 실습 자료입니다.



두 파트로 진행

오늘의 실습

- | | | |
|----------|------------------------|---|
| ● PART 1 | 누리온 환경 준비 | 접속 → 디렉터리 → 모듈 → pseudo potential → 배치 스크립트 |
| ● PART 1 | QE 입력 파일 이해하기 | 범용 템플릿 한 장씩 짚기 |
| ● PART 1 | calculation 3종 비교 | scf · relax · vc-relax 무엇을 고정하고 무엇을 푸나 |
| ● PART 1 | vc-relax — 평형 격자 찾기 | Relaxed 구조를 먼저 잡고 이후 계산의 시작점으로 |
| ● PART 2 | SCF — 전체 에너지 계산 | 최적 격자 위에서 전체 에너지·힘·응력 |
| ● PART 2 | NSCF + bands.x / dos.x | 밴드 구조와 상태 밀도, plotband.x + gnuplot으로 시각화 |
| ● PART 2 | 출력 확인과 자주 겪는 함정 | grep 한 줄로 수렴 확인, 함정 5가지 |



누리온 환경 준비

접속, 모듈 로드, 배치 스크립트까지



이 실습에서 알아야 할 것만

누리온 한눈에

두 가지 파일 시스템

/home01/\$USER — 64GB, 소스 파일·스크립트 보관용
→ **cd** 명령으로 이동

/scratch/\$USER — 100TB, 실제 계산 작업 공간
→ **cds** 명령으로 이동
※ 15일 미접근 파일 자동 삭제
※ 작업 제출은 **여기서만 가능**

실습용 KISTI 슈퍼컴퓨터

노드 종류 KNL (Intel Xeon Phi)
노드당 코어 수 68 cores
최대 작업 시간 12시간 (실습엔 충분)
대기 시간 **거의 없음**



한 줄씩 그대로 따라 입력

누리온 접속과 작업 디렉터리 만들기

TERMINAL

1. Termius SSH로 누리온 접속

Termius에 등록한 Host로 누리온 접속 or `$ ssh <your_id>@nurion.ksc.re.kr`

OTP: *****/ Password: *****/

2. /scratch로 이동 (모든 계산은 반드시 여기서)

`$ cds` `# cd /scratch/$USER` 와 같은 동작

3. 실습용 작업 디렉터리 생성

`git clone https://github.com/YoonsangTak/TEM_Workshop_2026`

4. 기초 실습 디렉토리 진입

`cd TEM_Workshop_2026/1_dft_basics/`

5. 현재 위치 확인

`$ pwd`

`/scratch/<your_id>/TEM_Workshop_2026/1_dft_basics` 나오는거 확인

<your_id>는 KISTI에서 발급받은 본인 계정 ID로 바뀌서 입력. /scratch 경로의 실제 마운트명은 시스템에 따라 다를 수 있음.



module 명령으로 컴파일러·MPI·QE 로드

QE 실행 환경 설정 (참고)

TERMINAL

1. 기존 모듈 모두 제거 (clean start)

\$ module purge

2. KNL 노드용 architecture

\$ module load craype-mic-knl

3. 컴파일러와 MPI

\$ module load intel/19.0.5

\$ module load impi/19.0.5

4. Quantum ESPRESSO 모듈

\$ module load qe/7.2

5. 로드된 모듈 확인

\$ module list

순서가 중요합니다

craype-mic-knl 가 가장 먼저

→ KNL 노드용 환경 설정. 안 하면 SKL용 바이너리가 로드되어 KNL에서 안 돌아감

intel, impi 동일 버전으로

→ QE 모듈이 어떤 컴파일러로 빌드되었는지에 맞춰야 함

qe/7.2 (default) 또는 원하는 버전

→ \$ module av qe 로 설치된 버전 확인 가능



Si.pbe-n-rrkjus_psl 한 개만 있으면 충분

Pseudo-potential 파일 받기 (참고)

DOWNLOAD

```
# pseudo/ 디렉터리로 이동  
cd pseudo
```

```
# PSlibrary에서 Si PBE ultrasoft PP 다운로드  
wget https://pseudopotentials.quantum-espresso.org/upf_files/Si.pbe-n-rrkjus_psl.1.0.0.UPF
```

Pseudo-potential 이란?

원자의 안쪽 전자들을 평균화한 효과적 퍼텐셜. QE 계산에 반드시 필요한 외부 파일로, 원소마다 따로 받아야 합니다. PSlibrary, SSSP, Pseudo Dojo 등에서 공식 PP를 받을 수 있습니다.

PP 파일 이름 읽기

Si · 원소 기호
pbe · 교환상관 범함수 (PBE GGA)
rrkjus · ultrasoft 종류 (RRKJ)
psl · PSlibrary 소스



run_qe.sh — 모든 계산에 공통으로 사용

작업 제출 스크립트 작성

run_qe.sh

```
#!/bin/sh
#PBS -N qe_si
#PBS -V
#PBS -q normal
#PBS -A qe
#PBS -l select=1:ncpus=68:mpiprocs=16:ompthreads=1
#PBS -l walltime=01:00:00
```

```
cd $PBS_O_WORKDIR
```

```
module purge
module load craype-mic-knl
module load intel/19.0.5 impi/19.0.5
module load qe/7.2
```

계산 실행 — INPUT_FILE만 바뀌가며 재사용

```
INPUT=si.vc-relax.in
OUTPUT=${INPUT%.in}.out
mpirun -np 16 pw.x -in $INPUT > $OUTPUT
```

PBS 옵션 풀이

-q normal

큐 이름. 간단한 확인은 debug 사용, 본 계산은 normal (누리온 전용)

-A qe

App Name. QE는 'qe'.

select=1:ncpus=68:mpiprocs=16

노드 1개 점유, 68코어 할당, 그 중 MPI 프로세스 16개 사용

walltime=01:00:00

1시간 후 강제 종료. light 실습엔 충분.



qsub으로 던지고 qstat으로 확인

Queue system: 작업 제출과 모니터링

SUBMIT

```
# 작업 제출 (반드시 /scratch에서!)  
qsub run_qe.sh  
1234.pbs          # 작업 ID 반환
```

MONITOR

```
# 내 작업 상태 확인  
qstat -u $USER
```

Job id	Name	User	Time Use	S	Queue
1234.pbs	qe_si	myid	00:01:23	R	debug

R = Running
Q # Q = Queued (대기)
E # E = Exiting (종료중)

MISC

```
qdel 1234.pbs      # 작업 취소    $ ls -lt      # 출력 파일 확인
```



QE 입력 파일과 계산

이제 입력 파일을 만들고, 앞서 만든 `run_qe.sh`로 실제 계산을 돌려봅니다.



기본 3종 세트

어떤 .x 파일을 언제 쓰나?



pw.x

전체 에너지와 힘

Plane-Wave 메인 엔진.
SCF, NSCF, relax, vc-relax, MD까지 대부분이
여기서 출발.

```
mpirun pw.x -in input.in > out
```



bands.x

밴드 구조 후처리

NSCF 결과를 받아 k-path를 따라 밴드 데이터 정
리.
plotband.x로 시각화.

```
mpirun bands.x -in bands.in
```



dos.x

상태 밀도(DOS)

NSCF 결과로부터 전자 상태 밀도 계산.
projwfc.x로 PDOS도 가능.

```
dos.x -in dos.in
```



대부분의 시스템에서 사용 가능

범용 입력 템플릿 (1/2)

input.in (1/2)

&CONTROL

```
calculation = 'scf'
prefix      = 'si'
outdir      = './tmp/'
pseudo_dir  = './pseudo/'
verbosity   = 'default'
restart_mode = 'from_scratch'
nstep       = 50
forc_conv_thr = 1.0d-3
etot_conv_thr = 1.0d-4
```

/

&SYSTEM

```
ibrav = 0,
nat   = 2,
ntyp  = 1
ecutwfc = 25.0
ecutrho = 200.0
occupations = 'smearing'
smearing    = 'mv'
degauss     = 0.01
nspin       = 1
! lda_plus_u = .true.
! Hubbard_U(1) = 4.0
```

/

(뒤 슬라이드에 이어서...)

주요 parameter 의미

calculation — scf / nscf / bands / relax / vc-relax / md

nstep — relax 최대 단계 수

ibrav — 2=fcc, 4=hex, 0=CELL_PARAMETERS 명시

ecutwfc / ecutrho — 절단 에너지(Ry). ultrasoft면 비율 8배

occupations + smearing — Metallic에서만, 절연체는 'fixed'

nspin — spin up down 구분 여부. 2 = 자성

Hubbard_U — 전이금속 산화물(Ni, Mn, Co, Fe ...)



수렴 방식과 카드

범용 입력 템플릿 (2/2)

input.in (2/2)

&ELECTRONS

```
conv_thr      = 1.0d-6
mixing_mode    = 'plain'
mixing_beta    = 0.4
diagonalization = 'cg'
electron_maxstep = 100
```

&IONS

```
ion_dynamics = 'bfgs'
```

&CELL

```
cell_dynamics = 'bfgs'
press         = 0.0
cell_dofree   = 'all'
```

ATOMIC_SPECIES

```
Si 28.0855 Si.pbe-n-rrkjus_psl.1.0.0.UPF
```

CELL_PARAMETERS angstrom

```
0.00000000 2.71500000 2.71500000
2.71500000 0.00000000 2.71500000
2.71500000 2.71500000 0.00000000
```

ATOMIC_POSITIONS crystal

```
Si 0.00 0.00 0.00
Si 0.25 0.25 0.25
```

K_POINTS automatic

```
6 6 6 1 1 1
```

https://www.quantum-espresso.org/Doc/INPUT_PW.html

주요 parameter 의미

conv_thr — SCF 수렴 임계 (Ry)

mixing_beta — 수렴이 안 되면 값을 줄여보기

diagonalization — david 빠름, cg는 안정·메모리 적음

ion_dynamics = 'bfgs'

→ 표준 relax. md는 'damp'

cell_dofree — 'all' (전체), 'xy' (in plane 길이만 변화), 'volume' (모양 고정)

ATOMIC_POSITIONS alat

→ alat / crystal / angstrom / bohr 단위 선택

K_POINTS automatic

→ Monkhorst-Pack 격자 + offset(1=shift)



calculation 키워드 3가지

무엇을 고정하고 무엇을 푸나?

calculation = 'scf'

에너지만

고정 원자 위치 · 격자

계산 전자 밀도, 에너지

이미 구조가 정해진 상태에서
에너지 · 힘 · 응력을 한 번 본다

calculation = 'relax'

원자만 풀기

고정 격자 (셀 크기 · 모양)

최적화 원자 위치

셀은 그대로 두고
내부 원자만 평형 위치로 이완

calculation = 'vc-relax'

전부 풀기

고정 없음

최적화 원자 위치 + 격자 변수

DFT 평형 격자상수를 찾는다
→ 이후 모든 계산의 시작점

vc-relax로 평형 격자를 먼저 잡고, 그 구조 위에서 scf와 후처리를 진행합니다.



vc-relax 입력 파일 (실습용 light 설정)

평형 격자상수 찾기

si.vc-relax.in

&CONTROL

```
calculation = 'vc-relax'
prefix      = 'si'
outdir      = './tmp/'
pseudo_dir  = './pseudo/'
forc_conv_thr = 1.0d-3
etot_conv_thr = 1.0d-4
```

&SYSTEM

```
ibrav = 0
nat = 8
ntyp = 1
ecutwfc = 40.0
ecutrho = 320.0
```

&ELECTRONS

```
conv_thr = 1.0d-6
```

&IONS

&CELL

```
cell_dynamics = 'bfgs'
```

```
press = 0.0
```

ATOMIC_SPECIES

```
Si 28.0855 Si.pbe-n-rrkjus_psl.1.0.0.UPF
```

CELL_PARAMETERS angstrom

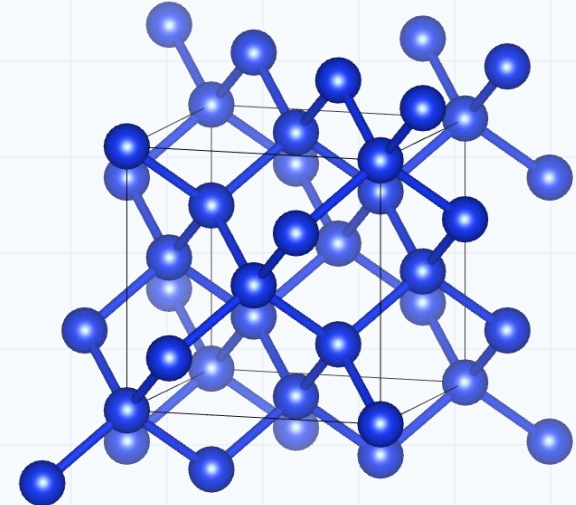
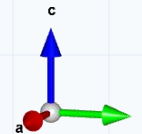
```
5.43 0.00 0.00
0.00 5.43 0.00
0.00 0.00 5.43
```

ATOMIC_POSITIONS crystal

```
Si 0.00 0.00 0.00
Si 0.50 0.50 0.00
Si 0.50 0.00 0.50
Si 0.00 0.50 0.50
Si 0.25 0.25 0.25
Si 0.75 0.75 0.25
Si 0.75 0.25 0.75
Si 0.25 0.75 0.75
```

K_POINTS automatic

```
4 4 4 1 1 1
```



RUN

```
# run_qe.sh의 INPUT만 바꿔서 제출
```

```
$ vi run_qe.sh
```

```
→ INPUT=si.vc-relax.in
```

```
$ qsub run_qe.sh
```

```
$ qstat -u $USER # 진행 확인
```



출력 끝부분 'Begin final coordinates'

최적화된 격자와 좌표 꺼내기

EXTRACT

```
grep -A 20 'Begin final coordinates' si.vc-relax.out | tail -30
```

OUTPUT

Begin final coordinates

new unit-cell volume = $\langle \text{your_volume} \rangle \text{ a.u.}^3$

CELL_PARAMETERS

5.4680 0.0000 0.0000
0.0000 5.4680 0.0000
0.0000 0.0000 5.4680

ATOMIC_POSITIONS

Si 0.000000000 0.000000000 0.000000000
Si 0.250000000 0.250000000 0.250000000
...

End final coordinates

위 CELL_PARAMETER의 대각 성분의 값을 Bohr radius로 변환한 $\langle \text{your_alat} \rangle$ 값을 다음 SCF 입력의 celldm(1) 자리에 적습니다
혹은 celldm(1)이 아니라 A 로 태그를 적으면 angstrom 단위로 적을 수 있습니다.
production 값과 약간 다를 수 있습니다



vc-relax에서 얻은 $\langle \text{your_alat} \rangle$ 을 그대로 사용

최적 격자 위에서 SCF

si.scf.in

&CONTROL

```
calculation = 'scf'
prefix      = 'si'
outdir      = '../tmp/'
pseudo_dir  = '../pseudo/'
tpnfor = .true., tstress = .true.
```

/

&SYSTEM

```
ibrav = 2, celldm(1) =  $\langle \text{your\_alat} \rangle$ 
nat = 2, ntyp = 1
ecutwfc = 40
```

/

&ELECTRONS

```
conv_thr = 1.0d-6
```

/

ATOMIC_SPECIES

```
Si 28.0855 Si.pbe-n-rrkjus_psl.1.0.0.UPF
```

ATOMIC_POSITIONS alat

```
Si 0.00 0.00 0.00
Si 0.25 0.25 0.25
```

K_POINTS automatic

```
12 12 12 1 1 1
```

vc-relax에서 바뀐 값

$\text{celldm}(1) = \langle \text{your_alat} \rangle$

vc-relax 출력의 CELL_PARAMETERS에서 읽은 값

tpnfor / tstress = .true. 추가
잔여 stress와 힘을 함께 확인

K_POINTS를 증가시킴 (12×12×12)

RUN

```
$ vi run_qe.sh # INPUT=si.scf.in
$ qsub run_qe.sh
```



grep 한 줄이면 됩니다

출력에서 꼭 확인할 3가지

convergence

"achieved" 문구가 있는지

10^{-6} Ry

scf의 conv_thr 도달 여부

JOB DONE

마지막에 정상 종료 표시

CHECK

한 번에 핵심 정보만 추출

```
grep -E '!|convergence has been achieved|JOB DONE|Total force|P=' si.scf.out
```

convergence has been achieved in <N> iterations

```
! total energy      = <your_energy> Ry
Total force = <your_force> Total SCF correction = ...
total stress (Ry/bohr**3) ... P= <your_pressure>
JOB DONE.
```



k -path를 따라 $\epsilon(k)$

밴드 구조 계산

si.bands.in

```
&CONTROL
calculation = 'bands'
prefix      = 'si'
outdir      = '../tmp/'
pseudo_dir  = '../pseudo/'
/
&SYSTEM
ibrav = 2, celldm(1) = <your_alat>
nat = 2, ntyp = 1
ecutwfc = 40
nbnd = 8
/
&ELECTRONS
conv_thr = 1.0d-6
/
ATOMIC_SPECIES
Si 28.0855 Si.pbe-n-rrkjus_psl.1.0.0.UPF
ATOMIC_POSITIONS alat
Si 0.00 0.00 0.00
Si 0.25 0.25 0.25
K_POINTS crystal_b
5
0.500 0.500 0.500 30 ! L
0.000 0.000 0.000 30 ! Gamma
0.500 0.000 0.500 30 ! X
0.625 0.250 0.625 30 ! U
0.000 0.000 0.000 0  ! Gamma
```

si.bands_pp.in

```
&BANDS
prefix = 'si'
outdir = './tmp/'
filband = 'si.bands.dat'
/
```

RUN

```
# 1단계: scf 결과 기반 bands 계산
$ vi run_qe.sh # INPUT=si.bands.in
$ qsub run_QE.sh

# 2단계: bands.x로 밴드 데이터 정리
$ qsub run_bandx.sh
#!/bin/sh
#PBS -N si_bands_pp
#PBS -q debug
#PBS -A qe
#PBS -l select=1:ncpus=68:mpiprocs=16:omphreads=1
#PBS -l walltime=00:10:00

cd $PBS_O_WORKDIR
module purge
module load craype-mic-knl intel/19.0.5 impi/19.0.5 qe/7.2
mpirun -np 16 bands.x -in si.bands_pp.in > si.bands_pp.out
```

k -path를 따라 각 k 점에서 고유값 $\epsilon(k)$ 만 계산
scf 계산을 통해 얻은 charge density는 고정

$\text{celldm}(1) = \langle \text{your_alat} \rangle$
vc-relax 출력의 CELL_PARAMETERS에서 읽은 값

K_POINTS를 Brillouin zone 따라 그려야 함



gnuplot으로 Bands 그리기

결과 그리기

BANDS (gnuplot)

\$ gnuplot

set terminal pngcairo enhanced size 800,600

set output 'si_bands.png'

set ylabel "Energy (eV)"

set yrange [-6:16]

EF = 5.9409

L = 0.0000

G1 = 0.8660

X = 1.8660

U = 2.2196

G2 = 3.2802

set xrange [L:G2]

set xtics ("L" L, "{/Symbol G}" G1, "X" X, "U" U, "{/Symbol G}" G2)

set grid xtics lt 1 lc rgb "gray" lw 1

set arrow from graph 0, first EF to graph 1, first EF nohead lc rgb "red" lw 1

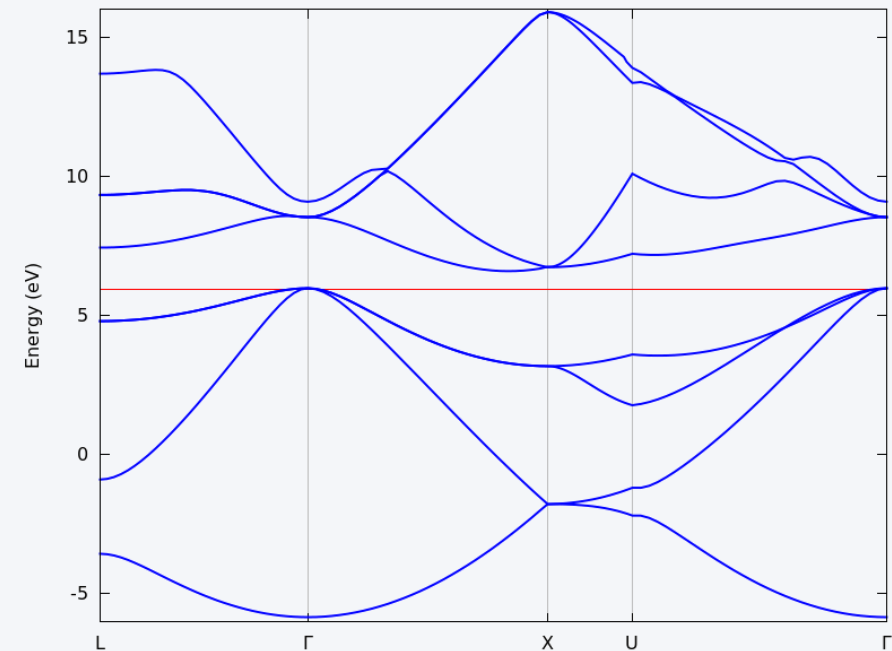
plot 'si.bands.dat.gnu' u 1:2 w l lw 2 lc rgb "blue" notitle

set output

exit

← `grep 'high-symmetry' si.bands_pp.out | head -1`
`grep -i 'highest occupied' si.scf.out`

\$ cp si_bands.png ~/1_dftbasics/





균일한 k-격자로 NSCF 후 dos.x

상태 밀도 계산

si.nscf.in

&CONTROL

```
calculation = 'nscf'
prefix      = 'si'
outdir      = './tmp/'
pseudo_dir  = './pseudo/'
```

&SYSTEM

```
ibrav = 2, celldm(1) = <youralat>
nat = 2, ntyp = 1
ecutwfc = 40
occupations = 'tetrahedra'
```

&ELECTRONS

```
conv_thr = 1.0d-6
```

ATOMIC_SPECIES

```
Si 28.0855 Si.pbe-n-rrkjus_psl.1.0.0.UPF
```

ATOMIC_POSITIONS alat

```
Si 0.00 0.00 0.00
Si 0.25 0.25 0.25
```

K_POINTS automatic

```
12 12 12 0 0 0
```

si.dos.in

&DOS

```
prefix = 'si'
outdir = './tmp/'
fildos = 'si.dos.dat'
Emin = -10.0, Emax = 20.0
DeltaE = 0.05
```

RUN

1단계: nscf (균일 k-격자)

```
$ vi run_qe.sh
```

```
$ qsub run_qe.sh
```

2단계: dos.x

```
$ vi dosx.sh
```

```
#!/bin/sh
```

```
#PBS -N si_bands_pp
```

```
#PBS -q debug
```

```
#PBS -A qe
```

```
#PBS -l select=1:ncpus=68:mpiprocs=16:ompthreads=1
```

```
#PBS -l walltime=00:10:00
```

```
cd $PBS_O_WORKDIR
```

```
module purge
```

```
module load craype-mic-knl intel/19.0.5 impi/19.0.5 qe/7.2
```

```
mpirun -np 16 dos.x -in si.dos.in > si.dos.out
```



gnuplot으로 DOS 그리기

결과 그리기

DOS (gnuplot)

\$ gnuplot

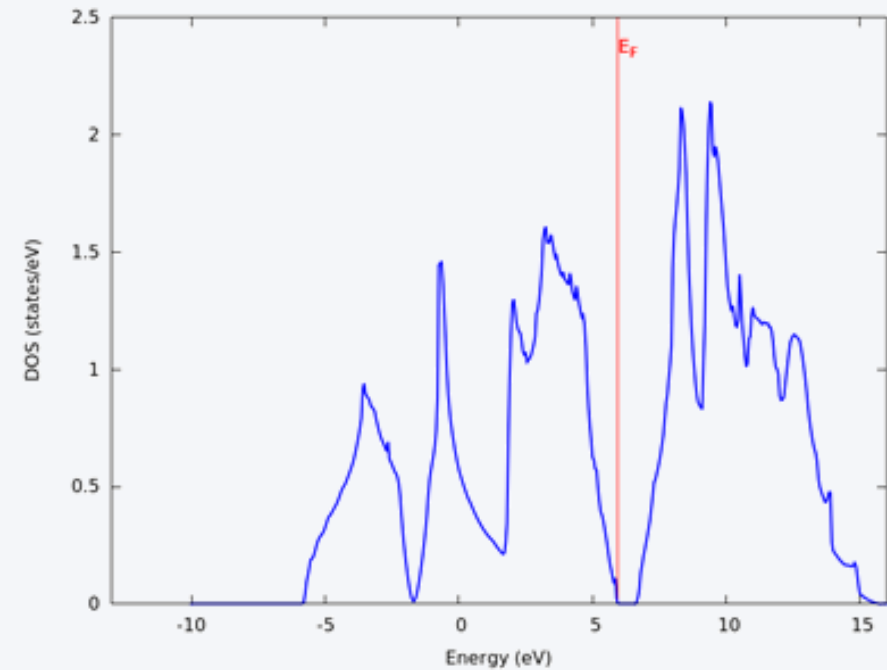
```
set terminal pngcairo enhanced size 800,600
set output 'si_dos.png'
set xlabel "Energy (eV)"
set ylabel "DOS (states/eV)"
set xrange [-13:16]
set yrange [0:*
```

EF = 5.9409

```
set arrow from EF, graph 0 to EF, graph 1 nohead lc rgb "red" lw 1 dt 2
set label "E_F" at EF, graph 0.95 tc rgb "red"
plot 'si.dos.dat' u 1:2 w l lw 2 lc rgb "blue" notitle
set output
exit
```

grep -i 'highest occupied' si.scf.out

\$ cp si_dos.png ~/1_dftbasics/





환경 준비부터 시각화까지

전체 파이프라인 한눈에



vc-relax 결과의 원자 구조를 이후 모든 단계의 input으로 동일하게 사용하여 전자 구조까지 계산